

DecisionsSearch — instalação e operação

Guia público para instalar o DecisionsSearch localmente, executar a composição suportada e verificar o servidor.

Estado suportado

O servidor HTTP/MCP atual opera no modo **full**, com:

- Neo4j para o ledger canônico, relações e contexto estrutural;
- Qdrant para a projeção de busca vetorial densa e, quando habilitada, sparse;
- JSONL apenas para landing zone, snapshots e estado operacional local.

mode: `light` permanece aceito como rótulo de compatibilidade de configuração, mas não ativa um servidor HTTP/MCP somente em JSONL. Para uma regressão sem infraestrutura, use os backends locais explícitos dos testes e benchmarks.

Pré-requisitos

- Python 3.11 ou superior;
- uv;
- Docker Desktop ou Docker Engine com Compose;
- uma senha local para NE04J_PASSWORD.

Instalação

```
git clone https://github.com/Renzo-Tognella/DecisionsSearch.git
cd DecisionsSearch
uv sync
cp .env.example .env
cp config/decisionssearch.yaml.example config/decisionssearch.yaml
```

Edite `.env` e defina `NE04J_PASSWORD`. As chaves de LLM, embeddings e integrações são opcionais para os fluxos que não as utilizam, mas devem ser configuradas antes de habilitar esses provedores.

Inicialização completa

Suba os serviços e aguarde os healthchecks:

```
docker compose up -d --wait --wait-timeout 120
```

O bootstrap prepara a coleção `memories` no Qdrant e o catálogo inicial no Neo4j. Ele pode ser executado novamente:

```
uv run python -m scripts.bootstrap
```

Se precisar apenas verificar ou criar a coleção vetorial, use o comando específico:

```
uv run python -m scripts.bootstrap_qdrant
```

Inicie o servidor HTTP + MCP:

```
uv run decisionssearch
```

O servidor escuta, por padrão, em `http://localhost:8000`.

Verificação

```
# Qdrant
curl -sf http://localhost:6333/healthz

# API HTTP
curl -sf http://localhost:8000/api/health

# MCP montado; sem headers MCP, 406 é uma resposta esperada de rota viva
curl -i http://localhost:8000/mcp/
```

Execute primeiro os testes que não dependem de serviços externos:

```
uv run pytest tests/ -q --ignore=tests/e2e
```

Os testes E2E exigem Neo4j e Qdrant ativos e só são habilitados explicitamente:

```
RUN_E2E=1 uv run pytest tests/e2e -q
```

Smoke test

O smoke test exercita criação e recuperação híbrida no caminho real. Ele grava uma memória de teste no projeto `ExampleProject` e não a remove ao terminar. Execute-o somente em uma base local descartável ou aceite esse registro:

```
uv run python scripts/smoke_e2e.py
```

Memória separada por projeto

Quando uma tool de memória não recebe `project`, o `DecisionsSearch` resolve a partição nesta ordem:

1. `DECISIONSSEARCH_PROJECT`, se definido;

2. o argumento `project` explícito;
3. o nome da raiz Git do workspace;
4. o nome da pasta atual quando não existe raiz Git.

O valor resolvido é gravado junto da memória. Na leitura, o projeto é filtrado no ledger, no Qdrant e no Neo4j antes da busca densa, sparse, estrutural e da fusão RRF. Essa separação é lógica: não substitui autenticação, autorização ou isolamento de infraestrutura.

Exemplo para um servidor iniciado fora do workspace do agente:

```
DECISIONSSEARCH_PROJECT=billing-service uv run decisionssearch
```

Parar e apagar dados locais

```
# Para os containers e mantém os volumes
docker compose down
```

```
# Para e apaga os volumes Neo4j/Qdrant; use somente se quiser perder os dados
docker compose down -v
```

Não publique `.env`, `config/decisionssearch.yaml`, `data/`, `.decisionssearch/`, caches, bancos locais ou corpus de avaliação privado.

Referências

- README em português e README em inglês;
- relatório técnico da memória;
- relatório público de resultados e limites.